

PMSCS 622P

Programming Languages

Lecture 03: Conditionals

Mr. Md. Masum Bhuiyan

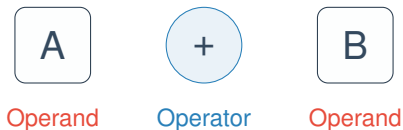
Department of Computer Science and Engineering
Jahangirnagar University

Lecture Roadmap

- 1 **Operators:** The tools to manipulate data.
- 2 **Expressions:** Combining variables and operators.
- 3 **Evaluation:** How the computer calculates results.
- 4 **Conditionals (Logic):** Teaching the computer to make decisions.
 - If / Else
 - Nested Logic
 - Switch / Match Case

3.1 What is an Operator?

An **operator** is a symbol that tells the compiler to perform specific mathematical or logical manipulations.



Arithmetic Operators

Used for mathematical calculations.

Symbol	Name	Example
+	Addition	$10 + 5 = 15$
-	Subtraction	$10 - 5 = 5$
*	Multiplication	$10 * 5 = 50$
/	Division	$20 / 4 = 5$
%	Modulus (Remainder)	$10 \% 3 = 1$

Warning: The Division Trap

How different languages handle division represents a major difference.

C++ / Java (Integer Division)

If both numbers are integers, the result is an integer (decimal is truncated).

```
int a = 5 / 2;  
// Result: 2  
// (0.5 is lost forever)
```

Python (True Division)

Python automatically converts to float.

```
a = 5 / 2  
# Result: 2.5  
# (Python creates a float)
```

To get C++ behavior in Python, use `//` (Floor Div).

Increment and Decrement

Shortcuts for adding or subtracting 1.

C++ / Java:

- `x++` (Post-increment): Use value, then add.
- `++x` (Pre-increment): Add, then use value.

```
int x = 10;  
x++; // x is 11
```

Python:

- Python **does not** have `++` or `--`.
- You must use assignment shortcuts.

```
x = 10  
x += 1 # x is 11  
# x++ is SyntaxError
```

Relational Operators

These compare two values and return a Boolean (true/false).

Symbol	Meaning	Logic
>	Greater Than	5 > 3 is True
<	Less Than	5 < 10 is True
>=	Greater/Equal	5 >= 5 is True
<=	Less/Equal	4 <= 5 is True
==	Equality Check	5 == 5 is True
!=	Not Equal	5 != 3 is True

Note: = is Assignment. == is Questioning.

Logical Operators

Used to combine multiple conditions.

AND

(C++: `&&`, Py: `and`)

Both must be True.

OR

(C++: `||`, Py: `or`)

At least one True.

NOT

(C++: `!`, Py: `not`)

Reverses the logic.

A	B	A && B	A B
T	T	T	T
T	F	F	T
F	T	F	T
F	F	F	F

3.2 Precedence of Operators

Just like PEMDAS in Math, computers follow a strict hierarchy.

Level	Operators	Direction
1 (Top)	() Parentheses	Inner → Outer
2	++ - (Post/Pre), ! (Not)	Right → Left
3	* / % (Multiplication)	Left → Right
4	+ - (Addition)	Left → Right
5	< <= > >= (Relational)	Left → Right
6	== != (Equality)	Left → Right
7	&& (Logical AND)	Left → Right
8	(Logical OR)	Left → Right
9 (Low)	= += -= (Assignment)	Right → Left

Writing Expressions: Instruction to Code

Instruction (Human Logic)	Code Expression
Check if age is between 18 and 25.	<code>(age >= 18) && (age <= 25)</code>
Calculate average of a , b , and c .	<code>avg = (a + b + c) / 3.0;</code>
Is <code>num</code> an even number?	<code>(num % 2) == 0</code>
Is score valid (0 to 100) OR is user admin?	<code>((score >= 0) && (score <= 100)) isAdmin</code>

3.3 Evaluation Rules: How Computers Think

Computers do not solve equations "all at once." They use a **Stack-based** approach or **Expression Tree**.

The Golden Rules

- 1 **Scan** from Left to Right (usually).
- 2 **Solve** highest precedence operators first.
- 3 **Replace** the operation with its result.
- 4 **Repeat** until one value remains.

Evaluation Trace: Arithmetic

Expression: `result = 10 + 5 * 2 - 8 / 4;`

Precedence Check:

- 1 * and / (Tie! Go Left → Right)
- 2 + and - (Tie! Go Left → Right)
- 3 = (Last)

Step-by-Step Reduction:

- $10 + \underline{5 * 2} - 8 / 4$ (Multiply)
- $10 + 10 - \underline{8 / 4}$ (Divide)
- $\underline{10 + 10} - 2$ (Add)
- $\underline{20 - 2}$ (Subtract)
- `result = 18` (Assign)

Evaluation Trace: Logic

Expression: `bool x = (5 > 3) && (8 != 8) || (10 == 10);`

Reduction:

1 Parentheses First:

- `5 > 3 → true`
- `8 != 8 → false`
- `10 == 10 → true`

2 New Expression: `true && false || true`

3 AND (&&) before OR (||):

- `true && false → false`

4 Final OR:

- `false || true → true`

Optimization: Short-Circuit Evaluation

Computers are lazy (efficient).

AND (&&)

If the **Left side is False**, the computer stops.

`False && (Complex Calculation)`

Result is False. Right side never runs.

OR (||)

If the **Left side is True**, the computer stops.

`True || (Complex Calculation)`

Result is True. Right side never runs.

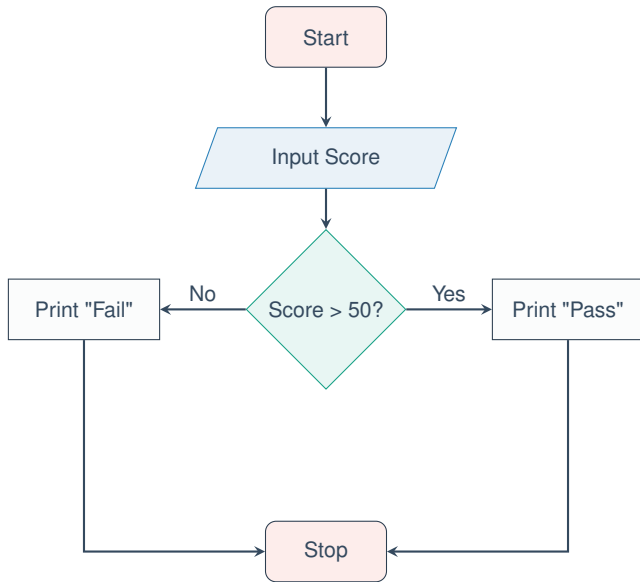
3.4 Introduction to Conditionals

Programs are not just straight lines. They need to make decisions.

- "If it rains, take an umbrella."
- "If password is correct, login."
- "If balance is low, reject payment."

This is achieved using **Control Flow** statements.

Logic Flowchart



The 'if' Statement

Executes code **only if** the condition is true.

C++ / Java

```
if (score > 50) {  
    // Body inside Braces  
    cout << "Pass";  
}
```

Python

```
if score > 50:  
    # Body indented  
    print("Pass")
```

Critical: Python uses Indentation (Spaces), C++ uses Braces {}.

The 'else' Statement

What to do if the condition is **False**?

C++ / Java

```
if (age >= 18) {  
    cout << "Adult";  
} else {  
    cout << "Minor";  
}
```

Python

```
if age >= 18:  
    print("Adult")  
else:  
    print("Minor")
```

Multiple Conditions (Else If)

Used to check more than two options.

C++ / Java (else if)

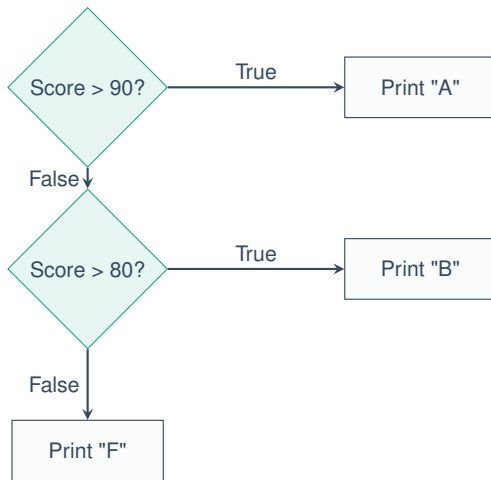
```
if (score >= 90) {  
    cout << "A";  
}  
else if (score >= 80) {  
    cout << "B";  
}  
else {  
    cout << "F";  
}
```

Python (elif)

```
if score >= 90:  
    print("A")  
elif score >= 80:  
    print("B")  
else:  
    print("F")
```

Visualizing the Ladder

The logic falls down until it finds a "bucket" that fits.



3.5 Nested Conditionals

An if statement inside another if statement.

Scenario: You need a ticket AND you need to be 18+.

```
if (hasTicket) {  
    if (age >= 18) {  
        cout << "Enter";  
    } else {  
        cout << "Too Young";  
    }  
} else {  
    cout << "Buy Ticket";  
}
```

Warning: Deeply nested code ("The Arrow Code") is hard to read.

Tip: Use Logical Operators (&&) to flatten nesting when possible.

Common Pitfalls

1. Assignment vs Comparison

`if (x = 5)` assigns 5 to `x` and returns `True`!
Use `if (x == 5)` for checking.

2. The Dangling Else

Indentation (Python) or Braces (C++) clarify which `if` the `else` belongs to.

```
// Ambiguous without braces  
if (a)  
    if (b) x++;  
else y++; // Does else belong to A or B? (It belongs to B!)
```

3.6 The Switch Statement

When testing one variable against many exact values, if-else is messy.

```
switch(day) {  
    case 1:  
        cout << "Monday";  
        break;  
    case 2:  
        cout << "Tuesday";  
        break;  
    default:  
        cout << "Invalid";  
}
```

Key Keywords:

- **case:** The value to match.
- **break:** Stops execution (Prevents "Fall-through").
- **default:** Like else, if no match found.

Python Match Case

Historically, Python did not have Switch. Python 3.10 introduced `match`.

```
# Python 3.10+
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case _:
        print("Invalid") # The underscore _ acts as Default
```

Before 3.10, Python programmers used Dictionaries or If-Elif ladders.

Understanding Fall-Through

In C++ and Java, if you forget `break`, code continues to the next case!

```
int x = 1;
switch(x) {
    case 1:
        cout << "One";
        // Missing break!
    case 2:
        cout << "Two";
        break;
}
```

Output:

Result

OneTwo

Sometimes useful (grouping cases), but usually a bug.

3.7 The Ternary Operator

A shortcut for simple If-Else assignment.

Syntax: `condition ? value_if_true : value_if_false;`

Regular If-Else:

```
if (age >= 18)
    type = "Adult";
else
    type = "Child";
```

Ternary Logic:

```
// C++ / Java
type = (age >= 18) ? "Adult" : "
    Child";
```

Python Version:

```
type = "Adult" if age >= 18 else
    "Child"
```

Practice: Leap Year Logic

Algorithm: A year is a leap year if:

- 1 Divisible by 4...
- 2 ...EXCEPT if divisible by 100...
- 3 ...UNLESS divisible by 400.

Complex Expression:

$(\text{year} \% 4 == 0) \text{ AND } ((\text{year} \% 100 \neq 0) \text{ OR } (\text{year} \% 400 == 0))$

Leap Year Code Implementation

```
int year = 2024;

if (year % 4 == 0) {
    if (year % 100 == 0) {
        if (year % 400 == 0) {
            cout << "Leap Year";
        } else {
            cout << "Not Leap Year";
        }
    } else {
        cout << "Leap Year";
    }
} else {
    cout << "Not Leap Year";
}
```

Can you simplify this using logical operators?

Simplified Leap Year

Using Logical Operators to remove nesting.

```
if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {  
    cout << "Leap Year";  
} else {  
    cout << "Not Leap Year";  
}
```

Lesson

Understanding **Operators** makes your **Conditionals** cleaner and faster.

Syntax Comparison Cheat Sheet

Feature	C++ / Java	Python
Block Start	Brace {	Colon :
Block End	Brace }	Un-indent
Else If	else if	elif
Logic AND	&&	and
Logic OR		or
Logic NOT	!	not
Switch	switch	match (v3.10+)

Class Exercises

- 1 Write an expression to check if a number is divisible by both 3 and 5.
- 2 Evaluate: `result = 10 * 2 + 5 > 20 && 1 == 1;`
- 3 Write an `if-else` ladder to categorize age:
 - 0-12: Child
 - 13-19: Teen
 - 20+: Adult
- 4 Convert the Age Logic into Python code.

Answer to Evaluation Exercise

Expression: $10 * 2 + 5 > 20 \ \&\& \ 1 == 1$

Steps:

- 1 Arithmetic: $10 * 2 \rightarrow 20$. Then $20 + 5 \rightarrow 25$.
- 2 Expression becomes: $25 > 20 \ \&\& \ 1 == 1$
- 3 Relational: $25 > 20 \rightarrow \text{true}$. $1 == 1 \rightarrow \text{true}$.
- 4 Logic: $\text{true} \ \&\& \ \text{true} \rightarrow \mathbf{\text{true}}$.

Lecture Summary

- **Operators** are the building blocks of logic.
- **Precedence** determines the order of calculation (PEMDAS).
- **Conditionals** allow programs to branch and make decisions.
- **Nesting** is powerful but dangerous (hard to read).
- Always consider **Syntax Differences** between C++ and Python (Braces vs Indentation).

Next Lecture: Loops